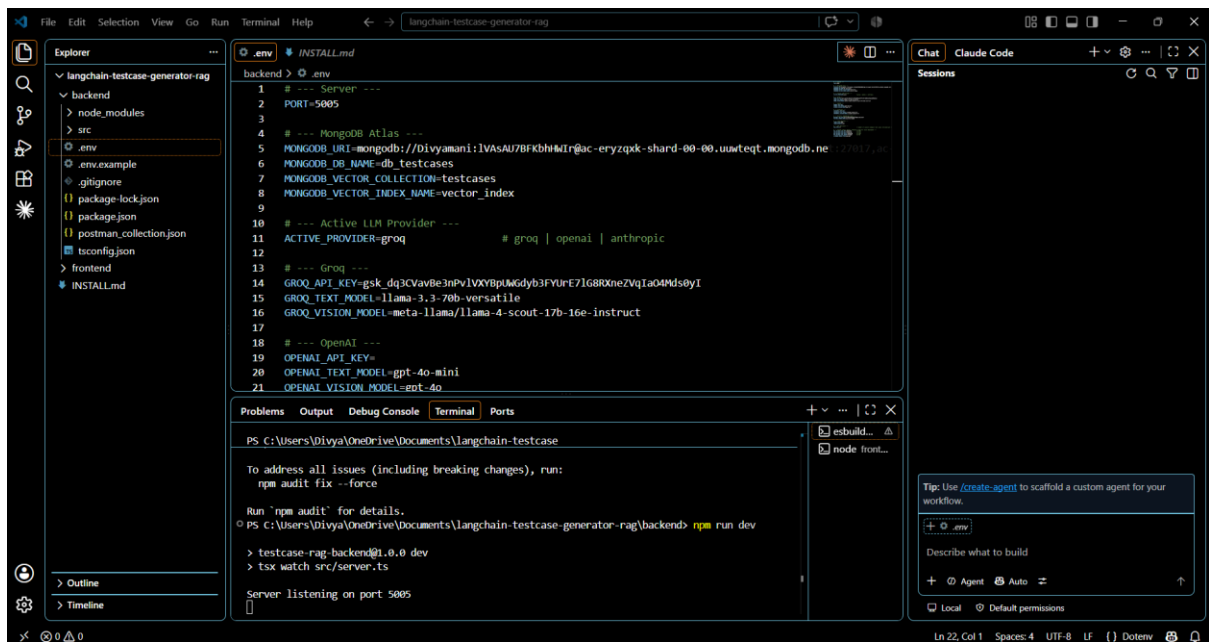


Langchain Testcase generator Installation



```
1 # --- Server ---
2 PORT=5005
3
4 # --- MongoDB Atlas ---
5 MONGODB_URI=mongodb://divyamani:IVASAU7BFkbhMIR@ac-eryzqxk-shard-00-00.uuwteq.mongodb.net:27017?authSource=admin
6 MONGODB_DB_NAME=db_testcases
7 MONGODB_VECTOR_COLLECTION=testcases
8 MONGODB_VECTOR_INDEX_NAME=vector_index
9
10 # --- Active LLM Provider ---
11 ACTIVE_PROVIDER=groq # groq | openai | anthropic
12
13 # --- Groq ---
14 GROQ_API_KEY=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
15 GROQ_TEXT_MODEL=llama-3.3-70b-versatile
16 GROQ_VISION_MODEL=meta-llama/llama-4-scout-17b-16e-instruct
17
18 # --- OpenAI ---
19 OPENAI_API_KEY=
20 OPENAI_TEXT_MODEL=gpt-4o-mini
21 OPENAI_VISION_MODEL=gpt-4o
```

PS C:\Users\Divya\OneDrive\Documents\langchain-testcase-generator-rag\backend> npm run dev

testcase-rag backend@1.0.0 dev
tsx watch src/server.ts

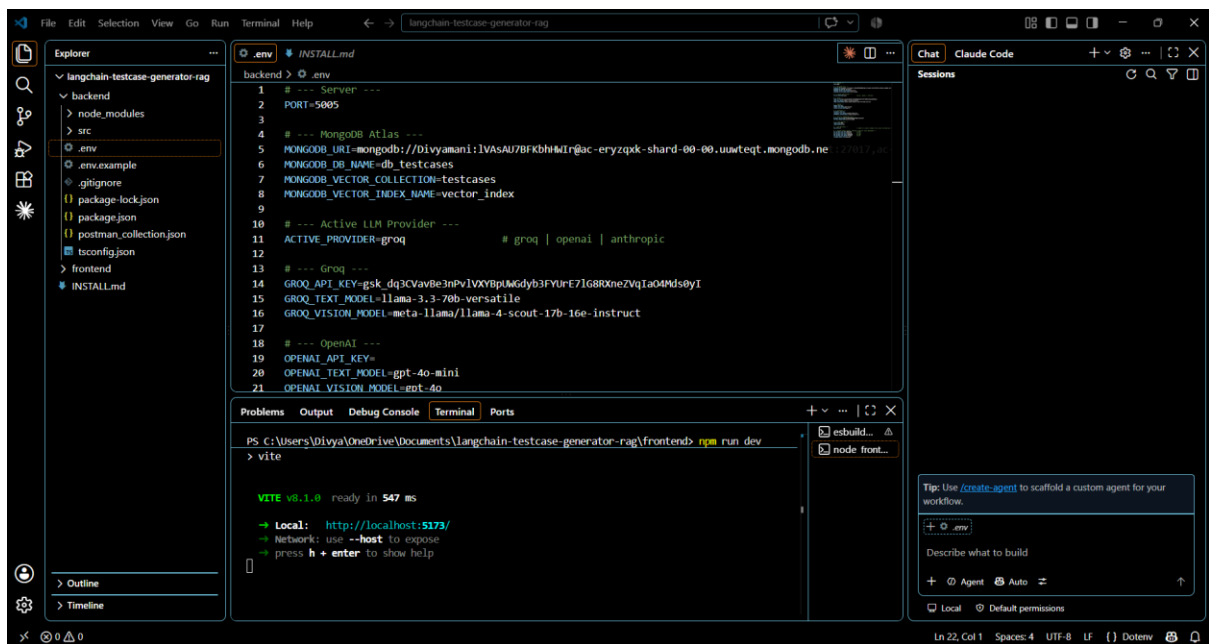
Server listening on port 5005

Tip: Use `/create-agent` to scaffold a custom agent for your workflow.

+ `env`

Describe what to build

+ Agent Auto



```
1 # --- Server ---
2 PORT=5005
3
4 # --- MongoDB Atlas ---
5 MONGODB_URI=mongodb://divyamani:IVASAU7BFkbhMIR@ac-eryzqxk-shard-00-00.uuwteq.mongodb.net:27017?authSource=admin
6 MONGODB_DB_NAME=db_testcases
7 MONGODB_VECTOR_COLLECTION=testcases
8 MONGODB_VECTOR_INDEX_NAME=vector_index
9
10 # --- Active LLM Provider ---
11 ACTIVE_PROVIDER=groq # groq | openai | anthropic
12
13 # --- Groq ---
14 GROQ_API_KEY=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
15 GROQ_TEXT_MODEL=llama-3.3-70b-versatile
16 GROQ_VISION_MODEL=meta-llama/llama-4-scout-17b-16e-instruct
17
18 # --- OpenAI ---
19 OPENAI_API_KEY=
20 OPENAI_TEXT_MODEL=gpt-4o-mini
21 OPENAI_VISION_MODEL=gpt-4o
```

PS C:\Users\Divya\OneDrive\Documents\langchain-testcase-generator-rag\frontend> npm run dev

vite

VITE v5.1.8 ready in 547 ms

→ Local: http://localhost:5173/

→ Network: use --host to expose

→ press h + enter to show help

Tip: Use `/create-agent` to scaffold a custom agent for your workflow.

+ `env`

Describe what to build

+ Agent Auto

Execution and checking the API calls

The screenshot shows the Test Case Generator web application running in a browser. The application is titled "Test Case Generator" and is a "RAG-based multimodal chat test case generator". It has a domain dropdown set to "Banking" and an "Attach file" button. A "New Chat" button is also present. A chat input field contains the text "generate test cases for registration". Below the input, a list of 7 test cases is displayed, each with a title and a description. The test cases are:

- TC-001 Successful New Customer Account Registration with Valid KYC
- TC-002 Account Registration Failure due to Missing Mandatory Fields
- TC-003 Duplicate Customer Registration with Existing PAN
- TC-004 KYC Document Validation – Upload Corrupted File
- TC-005 OTP/2FA Failure – Incorrect OTP Entry
- TC-006 Edge Case – Date of Birth on Minimum Age Boundary (18 Years)
- TC-007 Regulatory Compliance – AML Screening Trigger on High-Risk Country Address

The right side of the screenshot shows the browser's network tab. A request is visible with the name "chat" and the type "Request Payload". The payload is a JSON object:

```
{
  "domain": "banking",
  "message": "generate test cases for registration"
}
```

The screenshot shows the Test Case Generator web application running in a browser. The application is titled "Test Case Generator" and is a "RAG-based multimodal chat test case generator". It has a domain dropdown set to "Banking" and an "Attach file" button. A "New Chat" button is also present. A chat input field contains the text "generate test cases for registration". Below the input, a list of 7 test cases is displayed, each with a title and a description. The test cases are:

- TC-001 Successful New Customer Account Registration with Valid KYC
- TC-002 Account Registration Failure due to Missing Mandatory Fields
- TC-003 Duplicate Customer Registration with Existing PAN
- TC-004 KYC Document Validation – Upload Corrupted File
- TC-005 OTP/2FA Failure – Incorrect OTP Entry
- TC-006 Edge Case – Date of Birth on Minimum Age Boundary (18 Years)
- TC-007 Regulatory Compliance – AML Screening Trigger on High-Risk Country Address

The right side of the screenshot shows the browser's network tab. A request is visible with the name "chat" and the type "Response". The response is a JSON object:

```
{
  "sessionId": "715fc07e-f902-46ce-89b4-e439c1c746b6",
  "domain": "banking",
  "sourceType": "document",
  "testCases": [
    {
      "id": "TC-001",
      "title": "Successful New Customer Account Registration with Valid KYC",
      "steps": [
        "Launch the banking portal.",
        "Log in with a staff user having account creation permissions.",
        "Navigate to the 'Customer Onboarding' module.",
        "Select 'Create New Account' and choose account type.",
        "Enter all mandatory customer details (full name, email, phone number, etc.).",
        "Upload valid KYC documents (PAN card PDF, Aadhar card PDF, etc.).",
        "Click 'Generate OTP' for mobile verification.",
        "Submit the registration form."
      ],
      "expectedResult": "Customer account is created, and a confirmation message is displayed."
    },
    {
      "id": "TC-002",
      "title": "Account Registration Failure due to Missing Mandatory Fields",
      "steps": [
        "Launch the banking portal."
      ],
      "expectedResult": "Registration fails due to missing mandatory fields."
    }
  ]
}
```

Postman API Collection

The screenshot shows the Postman interface with a collection named "Test Case Generator RAG - Backend". The selected item is "Health Check". The request is a GET to `{{baseUrl}}/api/health`. The response is a 200 OK with a JSON body:

```
{
  "status": "ok",
  "db": "connected",
  "timestamp": "2026-08-18T14:00:39.422Z"
}
```

The interface includes a sidebar with collections, environments, documents, specs, mocks, and flows. The top bar shows the user "Divya M" and various utility icons.

The screenshot shows the Postman interface with the same collection. The selected item is "Generate Test Cases - From Document Text". The request is a POST to `{{baseUrl}}/api/generate-testcases`. The response is a 200 OK with a JSON body:

```
{
  "domain": "banking",
  "sourceType": "document",
  "requirementsText": "Fund transfers above INR 50,000 require OTP validation. Daily transfer limit is INR 200,000.",
  "testCases": [
    {
      "id": "TC-001",
      "title": "Fund Transfer below OTP threshold - successful transaction",
      "steps": [
        "Launch the banking application",
        "Login with valid credentials",
        "Navigate to Funds Transfer > New Transfer",
        "Select source account and beneficiary details",
        "Enter transfer amount of INR 30,000 (below INR 50,000)",
        "Proceed to confirmation and submit the transfer"
      ],
      "expectedResult": "Transfer is processed without OTP prompt and a success message with transaction reference is displayed"
    },
    {
      "id": "TC-002",
      "title": "Fund Transfer at OTP threshold (INR 50,000) - no OTP required",
    }
  ]
}
```

The interface is consistent with the first screenshot, showing the same sidebar and top bar.

Divya M

Search

Test Case Generator RAG - Backend > Chat - Turn 1 (Generate)

POST `{{baseUrl}}/api/chat` Send

Docs Params Authorization Headers 10 Body Scripts Settings Cookies

Query Params

Key	Value	Description	Bulk Edit
Key	Value	Description	

Body Cookies Headers 8 Test Results

200 OK · 5.40 s · 4.44 KB Save Response

```
{
  "sessionId": "7d013557-9e53-43a7-ab9b-bfc15ad5964f",
  "domain": "insurance",
  "sourceType": "document",
  "testCases": [
    {
      "id": "TC-001",
      "title": "Submit claim within coverage limit - no underwriter flag",
      "steps": [
        "Launch the Insurance Management System",
        "Login with valid underwriter credentials",
        "Navigate to Claims > New Claim",
        "Select an active policy with a coverage limit of $50,000",
        "Enter claim amount of $30,000 and complete required claim details",
        "Submit the claim"
      ],
      "expectedResult": "Claim is processed automatically and no underwriter review flag is generated"
    },
    {
      "id": "TC-002"
    }
  ]
}
```

Standing by for launch!

Globals Vault Tools

Divya M

Search

Test Case Generator RAG - Backend > Chat - Turn 2 (Refine using same sessionId)

POST `{{baseUrl}}/api/chat` Send

Docs Params Authorization Headers 10 Body Scripts Settings Cookies

Query Params

Key	Value	Description	Bulk Edit
Key	Value	Description	

Body Cookies Headers 8 Test Results

200 OK · 5.94 s · 5.01 KB Save Response

```
{
  "sessionId": "7d013557-9e53-43a7-ab9b-bfc15ad5964f",
  "domain": "insurance",
  "sourceType": "document",
  "testCases": [
    {
      "id": "TC-001",
      "title": "Submit claim within coverage limit - no underwriter flag",
      "steps": [
        "Launch the Insurance Management System",
        "Login with valid underwriter credentials",
        "Navigate to Claims > New Claim",
        "Select an active policy with a coverage limit of $50,000",
        "Enter claim amount of $30,000 and complete required claim details",
        "Submit the claim"
      ],
      "expectedResult": "Claim is processed automatically and no underwriter review flag is generated"
    },
    {
      "id": "TC-002"
    }
  ]
}
```

Standing by for launch!

Globals Vault Tools

Test Case Generator RAG - Backend > Chat - Validation Error (missing message)

POST `{{baseUrl}}/api/chat` Send

Docs Params Authorization Headers 10 Body Scripts Settings

Content-Type: application/json
Host
User-Agent
Accept
Accept-Encoding
Connection
Content-Type

Key Value Description

Body Cookies Headers 8 Test Results

400 Bad Request - 8 ms - 307 B Save Response

```
{
  "error": "message is required"
}
```

Connect Git Terminal Console Import Complete Add missing message field

Test Case Generator RAG - Backend > Generate Test Cases - Hospital Domain

POST `{{baseUrl}}/api/generate-testcases` Send

Docs Params Authorization Headers 10 Body Scripts Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON

Schema Beautify

```
1 {
2   "domain": "hospital",
3   "requirementsText": "On admission, every patient must be assigned a bed only after triage vitals are recorded. Prescriptions above the
4   standard dosage threshold require pharmacist countersignature before dispensing."
5 }
6
7 {
8   "id": "TC-001",
9   "title": "Assign Bed After Successful Triage Vitals Entry (Positive Admission Flow)",
10  "steps": [
11    "Launch the Hospital Information System application.",
12    "Login with valid clinician credentials.",
13    "Navigate to Patient Service -> Admission.",
14    "Select a new patient and open the Triage module.",
15    "Enter all required vital signs (e.g., Heart Rate, Respiratory Rate, Blood Pressure, SpO2, AVPU/GCS, Pain Score).",
16    "Submit the triage form and verify the success confirmation.",
17    "Proceed to the Bed Management tab.",
18    "Select an available ward bed and confirm the assignment.",
19    "Observe the system response."
20  ],
21  "expectedResult": "Bed is successfully assigned to the patient and a confirmation message \"Bed assigned\" is displayed; audit log
22  records triage completion followed by bed allocation."
23 }
```

Body Cookies Headers 8 Test Results

200 OK - 6.19 s - 5.23 KB Save Response

Connect Git Terminal Console Import Complete Standing by for launch!